

Especificação Técnica — Site Institucional Fernando Gonçalves

1. Objetivo do projeto

Desenvolver o site institucional de **Fernando Gonçalves** com uma estética moderna, sofisticada e premium, utilizando como referência visual o site atual:

<https://fernando-goncalves.vercel.app/>

O novo projeto deve priorizar:

- Alta performance;
- Carregamento extremamente rápido;
- Excelente experiência em dispositivos móveis;
- SEO técnico;
- Acessibilidade;
- Animações sofisticadas, porém leves;
- Design contemporâneo;
- Glassmorphism utilizado de forma controlada e elegante;
- Código organizado e fácil de manter;
- Arquitetura preparada para crescimento futuro.

O site é um **site institucional**, e não uma aplicação web complexa. Portanto, a arquitetura deve evitar JavaScript desnecessário.

2. Stack recomendada

Framework principal

Astro + TypeScript

Justificativa

Astro deve ser o framework principal porque o projeto é predominantemente institucional e orientado a conteúdo.

A arquitetura do Astro permite gerar HTML estático/pré-renderizado e carregar JavaScript somente nos componentes que realmente precisam de interatividade.

O objetivo é evitar transformar o site inteiro em uma aplicação JavaScript.

Princípio arquitetural

> HTML/CSS primeiro. JavaScript somente quando houver necessidade real.

3. Comparação com Next.js

Next.js é excelente e pode ser utilizado em projetos institucionais, porém não é necessário adotar React como arquitetura principal neste projeto.

Critério Astro Next.js
--- ---: ---:
Site institucional Excelente Excelente
Performance Excelente Muito alta
SEO Excelente Excelente
Baixo uso de JavaScript Excelente Bom
Animações Excelente Excelente
Landing pages Excelente Excelente
Blog Excelente Excelente
Aplicação complexa Muito bom Excelente

| Dashboard/SaaS | Bom | Excelente |

| Manter projeto extremamente leve | Excelente | Muito bom |

Para este projeto, escolher Astro como arquitetura principal.

4. Arquitetura de Islands

Utilizar a filosofia de **Islands Architecture** do Astro.

A maior parte do site deve permanecer como HTML/CSS estático.

Somente componentes que realmente precisarem de comportamento no cliente devem receber JavaScript.

Exemplo:

```text

Astro

|

├─ Header → Astro

├─ Hero → Astro

├─ Sobre → Astro

├─ Serviços → Astro

├─ Números → Astro

├─ Depoimentos → Astro

├─ CTA → Astro

├─ Footer → Astro

|

└─ Componente interativo específico → React/Svelte/JS

...

Não utilizar React por padrão em todos os componentes.

React, Vue, Svelte ou outra tecnologia de componente só deve ser introduzida quando houver justificativa técnica.

---

## # 5. Estrutura de pastas

Utilizar uma estrutura organizada semelhante a:

```text

src/

|

├─ components/

|

├─ Header.astro

|

├─ Hero.astro

|

├─ About.astro

|

├─ Numbers.astro

|

├─ Services.astro

|

├─ Testimonials.astro

|

├─ CTA.astro

|

└─ Footer.astro

|

├─ layouts/

|

└─ MainLayout.astro

|

├─ pages/

|

├─ index.astro

```
|   ├── sobre.astro
|   ├── palestras.astro
|   ├── contato.astro
|   └── ...
|
|   ├── styles/
|   |   ├── global.css
|   |   ├── tokens.css
|   |   └── animations.css
|   |
|   |   ├── scripts/
|   |   |   └── animations.ts
|   |   |
|   |   └── assets/
|   |       ├── images/
|   |       ├── icons/
|   |       └── fonts/
|   |
|   └── ...
```

A estrutura pode ser adaptada conforme a quantidade final de páginas.

Não criar pastas ou abstrações sem necessidade.

6. Sistema visual

A direção visual deve ser:

- Moderna;
- Premium;

- Editorial;
- Sofisticada;
- Minimalista;
- Tecnológica sem parecer um site de tecnologia genérico;
- Elegante;
- Com bastante respiro;
- Hierarquia visual clara.

Glassmorphism

Utilizar glassmorphism, mas de forma controlada.

Evitar o excesso de:

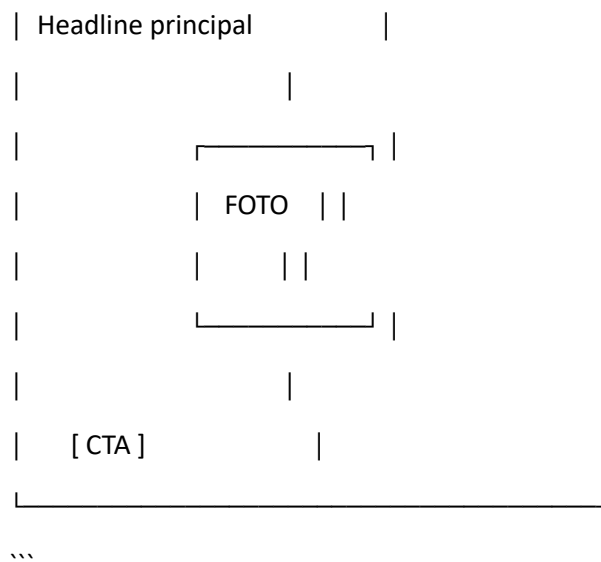
- Blur;
- Transparência;
- Gradientes;
- Brilhos;
- Sombras exageradas;
- Cards de vidro em todas as seções.

O glassmorphism deve funcionar como elemento de hierarquia visual.

Exemplo conceitual:

```text

|                    |  |
|--------------------|--|
|                    |  |
| HEADER             |  |
|                    |  |
| Fernando Gonçalves |  |
|                    |  |



Elementos flutuantes podem utilizar:

```
``css
background: rgba(...);
backdrop-filter: blur(...);
border: 1px solid rgba(...);
``
```

Porém, sempre preservar contraste, legibilidade e performance.

---

## # 7. Animações

O site deve ter animações, mas elas precisam ser **\*\*sofisticadas, discretas e performáticas\*\***.

A regra principal é:

> Não animar por animar.

As animações devem reforçar:

- Hierarquia;
- Direção visual;
- Feedback;
- Descoberta de conteúdo;
- Sensação premium;
- Continuidade entre seções.

---

## # 8. Hierarquia das tecnologias de animação

Utilizar as tecnologias nesta ordem:

### ## 8.1 CSS

Sempre que CSS resolver o problema, utilizar CSS.

Exemplos:

- Hover;
- Scale;
- Opacity;
- Transform;
- Transições;
- Underlines;
- Pequenos movimentos;
- Estados de botões;
- Microinterações.

Evitar JavaScript para efeitos que CSS consegue realizar.

---

## ## 8.2 Astro View Transitions

Utilizar para:

- Transições entre páginas;
- Entrada e saída de conteúdo;
- Continuidade de elementos;
- Navegação mais sofisticada.

---

## ## 8.3 GSAP ou Motion

Utilizar somente quando houver necessidade de animações mais complexas.

Exemplos:

- Hero cinematográfico;
- Sequências de entrada;
- Parallax controlado;
- Animações vinculadas ao scroll;
- Timeline complexa;
- Movimento coordenado de múltiplos elementos.

Não importar uma biblioteca pesada globalmente se apenas uma seção precisa dela.

Carregar o código de animação de forma localizada e quando possível somente quando necessário.

---

## # 9. Exemplo de comportamento do Hero

O Hero pode utilizar uma sequência semelhante a:

```text

Hero inicia



Imagem aparece suavemente



Headline entra



Texto secundário aparece



Elementos glassmorphism fazem pequeno deslocamento



CTA aparece



Durante o scroll, elementos têm pequeno movimento/parallax

```

As animações devem ser suaves e não comprometer a leitura.

Evitar animações longas ou excessivamente chamativas.

---

## # 10. Performance

Performance é um requisito fundamental do projeto.

### ## Regra geral

- > Se CSS consegue fazer, não usar JavaScript.
- >
- > Se JavaScript nativo consegue fazer, não adicionar uma biblioteca.
- >
- > Se uma biblioteca é necessária, carregá-la somente onde for necessária.

---

## # 11. Imagens

As imagens devem ser cuidadosamente otimizadas.

Priorizar:

1. AVIF;
2. WebP;
3. JPEG/PNG somente quando necessário.

Nunca utilizar uma imagem de vários megabytes sem otimização.

Utilizar:

- Diferentes dimensões para diferentes breakpoints;
- Lazy loading quando apropriado;
- `width` e `height` definidos para evitar layout shift;

- Imagem otimizada para o tamanho real de exibição;
- `fetchpriority="high"` apenas para imagens críticas, como o elemento visual principal do Hero.

A imagem principal do Hero deve ser tratada como recurso crítico.

Imagens abaixo da primeira dobra devem ser carregadas de forma lazy quando apropriado.

---

## # 12. Fontes

Evitar excesso de fontes e pesos.

Não utilizar:

`text`

8 pesos

4 famílias

20 arquivos

...

Preferir:

- Uma família principal;
- Uma família secundária apenas se houver necessidade real;
- Somente os pesos utilizados;
- Formatos modernos de fonte;
- `font-display: swap`.`

---

## # 13. JavaScript

Manter o JavaScript mínimo.

Não transformar todos os componentes em componentes client-side.

Evitar:

- Bibliotecas duplicadas;
- Dependências desnecessárias;
- Plugins para efeitos simples;
- Scripts globais que poderiam ser locais;
- Carregamento de código que não é utilizado na página.

Sempre avaliar o impacto de cada dependência no bundle final.

---

## # 14. CSS

Utilizar CSS moderno.

Priorizar:

- Custom Properties;
- `clamp()`;
- CSS Grid;
- Flexbox;
- Container Queries quando fizer sentido;
- `aspect-ratio`;

- `min()`;
- `max()`;
- `calc()`;
- Media queries bem estruturadas.

Evitar:

- CSS duplicado;
- estilos inline desnecessários;
- especificidade excessiva;
- hacks;
- valores arbitrários espalhados pelo projeto.

---

## # 15. Design Tokens

Centralizar as principais decisões visuais em `tokens.css`.

Exemplo conceitual:

```
``css
:root {
 --color-background: ...;
 --color-surface: ...;
 --color-text: ...;
 --color-muted: ...;
 --color-accent: ...;

 --radius-sm: ...;
 --radius-md: ...;
```

```
--radius-lg: ...;

--space-xs: ...;
--space-sm: ...;
--space-md: ...;
--space-lg: ...;
--space-xl: ...;

--container-max: ...;
}
...
```

Os valores devem ser definidos de acordo com o design final.

Não espalhar valores de cor, espaçamento e radius arbitrariamente pelo código.

---

## # 16. Responsividade

O site deve ser desenvolvido pensando primeiro na experiência real de uso, e não apenas em desktop.

Breakpoints devem ser definidos conforme o conteúdo exigir.

Priorizar:

- Desktop;
- Tablet;
- Mobile;
- Telas pequenas;

- Telas ultrawide quando necessário.

Não simplesmente reduzir o layout desktop.

O mobile deve possuir composição própria quando necessário.

---

## # 17. Mobile

No mobile:

- Evitar animações pesadas;
- Reduzir parallax;
- Reduzir quantidade de elementos decorativos;
- Garantir áreas de toque adequadas;
- Preservar hierarquia;
- Evitar textos excessivamente grandes;
- Evitar overflow horizontal;
- Garantir que o glassmorphism não prejudique legibilidade.

---

## # 18. Acessibilidade

Acessibilidade deve fazer parte da implementação desde o início.

Implementar:

- HTML semântico;
- Headings hierarquizados;

- `alt` adequado nas imagens;
- Links e botões semanticamente corretos;
- Navegação por teclado;
- Estados de foco visíveis;
- Contraste adequado;
- `aria-label` somente quando necessário;
- Respeito a `prefers-reduced-motion`.

Para usuários que preferem reduzir movimento, as animações devem ser reduzidas ou desativadas.

Exemplo:

```
```css
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```
```

---

## # 19. SEO técnico

Implementar SEO desde a arquitetura.

Cada página deve possuir:

- `title` único;
- Meta description;
- Canonical;
- Open Graph;
- Twitter/X Cards quando apropriado;
- HTML semântico;
- Estrutura correta de headings;
- URLs limpas;
- Sitemap;
- Robots.txt;
- Dados estruturados quando aplicáveis.

Não utilizar múltiplos H1 sem justificativa.

---

## # 20. Core Web Vitals

O projeto deve ser desenvolvido visando bons resultados em:

- LCP;
- CLS;
- INP.

Especial atenção para:

### LCP

O conteúdo principal do Hero deve aparecer rapidamente.

### CLS

Reservar espaço para:

- Imagens;
- Fontes;
- Componentes;
- Elementos dinâmicos.

Evitar elementos que aparecem posteriormente deslocando o conteúdo.

### INP

Evitar JavaScript excessivo e tarefas longas no navegador.

---

## # 21. Deploy

Manter o deploy na **Vercel**, caso seja conveniente para o projeto atual.

O projeto deve ser configurado para:

- Build otimizado;
- Cache adequado;
- Assets com cache eficiente;
- Compressão;
- CDN;
- HTTPS;

- Preview deployments.

---

## # 22. Arquitetura de componentes

Os componentes devem ser reutilizáveis, mas não criar abstrações prematuras.

Bom:

```
```text
```

Button

Card

Container

Section

Header

Footer

```
```
```

Evitar criar componentes genéricos excessivamente abstratos apenas para reduzir algumas linhas de código.

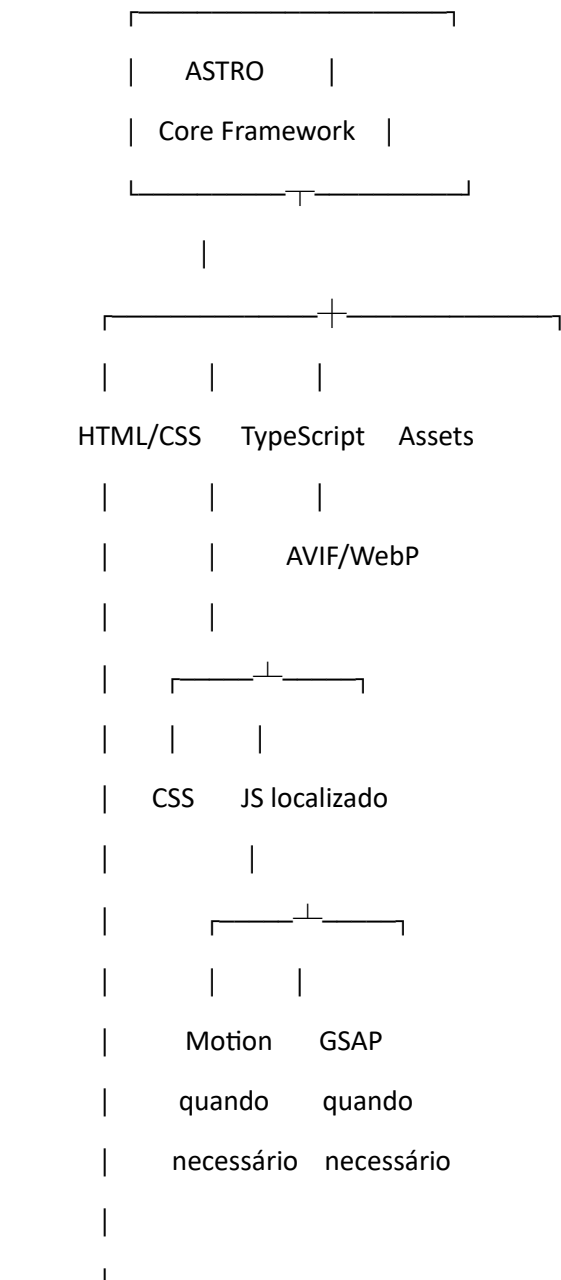
A arquitetura deve ser:

> reutilizável + legível + simples.

---

## # 23. Possível arquitetura final

```
```text
```



...

24. Princípios de desenvolvimento

Seguir estes princípios durante toda a implementação:

1. Performance first

Toda decisão visual deve considerar seu impacto no carregamento.

2. Progressive enhancement

O conteúdo principal deve funcionar sem depender de JavaScript.

3. Minimal JavaScript

Enviar ao navegador somente o JavaScript realmente necessário.

4. CSS first

Priorizar CSS para layout e microinterações.

5. Mobile first

Garantir excelente experiência em telas pequenas.

6. Acessibilidade by default

Não deixar acessibilidade para o final.

7. SEO by architecture

SEO deve ser incorporado à estrutura desde o início.

8. Design consistency

Utilizar tokens e componentes consistentes.

9. No unnecessary dependencies

Cada biblioteca deve possuir uma justificativa técnica.

10. No visual bloat

Glassmorphism, animações, sombras, blur e efeitos devem ser utilizados com moderação.

25. Critérios de aceitação

Antes de considerar o projeto concluído, verificar:

Visual

- [] Design premium;
- [] Glassmorphism elegante;
- [] Hierarquia visual clara;
- [] Espaçamento consistente;
- [] Tipografia sofisticada;
- [] Animações discretas e intencionais.

Performance

- [] Imagens otimizadas;
- [] Fontes otimizadas;
- [] JavaScript mínimo;
- [] Sem dependências desnecessárias;
- [] Lazy loading adequado;
- [] Layout sem mudanças inesperadas;

- [] Excelente performance mobile.

UX

- [] Navegação clara;
- [] CTAs identificáveis;
- [] Feedback visual;
- [] Estados hover/focus;
- [] Excelente experiência mobile;
- [] Sem scroll horizontal;
- [] Formulários fáceis de utilizar.

Acessibilidade

- [] HTML semântico;
- [] Navegação por teclado;
- [] Contraste adequado;
- [] Alt texts;
- [] Focus states;
- [] Reduced motion.

SEO

- [] Titles;
- [] Meta descriptions;
- [] Canonicals;
- [] Open Graph;
- [] Sitemap;
- [] Robots;
- [] Structured data quando aplicável.

26. Regra fundamental para a IA que irá implementar

****Não implemente simplesmente um site bonito.****

Implemente um site institucional de alta qualidade técnica.

O resultado deve equilibrar:

```text

DESIGN

+

UX

+

PERFORMANCE

+

SEO

+

ACESSIBILIDADE

+

MANUTENIBILIDADE

```

Nenhum desses aspectos deve ser sacrificado desnecessariamente em favor de outro.

O objetivo final é produzir um site que pareça sofisticado e tecnologicamente avançado para o usuário, mas que por baixo seja uma aplicação extremamente enxuta, organizada e rápida.

27. Decisão final de stack

Utilizar como padrão:

``text

Framework:

Astro

Linguagem:

TypeScript

Estilo:

CSS moderno

Animações básicas:

CSS

Transições de páginas:

Astro View Transitions

Animações complexas:

GSAP ou Motion, somente quando necessário

Imagens:

AVIF / WebP

Ícones:

SVG

Deploy:

Vercel

Arquitetura:

Astro Islands

Princípio:

Minimal JavaScript

...

Não utilizar por padrão

- React em toda a aplicação;
- Bibliotecas de animação globalmente;
- Bibliotecas de UI pesadas;
- Carrosséis desnecessários;
- JavaScript para efeitos que CSS resolve;
- Imagens sem otimização;
- Fontes e pesos excessivos;
- Glassmorphism aplicado indiscriminadamente.

28. Resultado esperado

O resultado deve ser um site institucional moderno e premium para Fernando Gonçalves, com:

- Visual contemporâneo;
- Glassmorphism sofisticado;
- Animações fluidas;
- Excelente responsividade;
- Carregamento rápido;
- SEO técnico sólido;

- Acessibilidade;
- Código limpo;
- Baixo consumo de JavaScript;
- Excelente experiência em desktop e mobile.

****Prioridade máxima:****

> Criar uma experiência visual sofisticada sem transformar o site em uma experiência pesada.

A tecnologia deve trabalhar a favor do design — e não o contrário.